

COM1025: Web and Database Systems

Module Introduction, Web, URL
HTML, JavaScript

(Slides credited to Dr. Manos Panaousis)

Joe Appleton

Week 7

A Question To Start

- Why is this course teaching both Databases and Web ?

A Question To Start

- Why is this course teaching both Databases and Web ?
 - ✓ Almost everyone will interact with Databases everyday, several times a day
 - ✓ Very few people know how to write SQL Queries
 - ✓ We need to provide some interface over the database to allow the right data to be extracted without the need to know SQL
 - ✓ Web applications are one of the most commonly used ways to interact with data in a Database

Structure for Weeks 7 - 10

	LECTURES
WEEK 7	An Internet Primer
WEEK 8	Networking , HTML forms , HTTP, COOKIES Styles, Templating, Forms, GDPR, Routing
WEEK 9	Tying everything together - The three-tier architecture
WEEK 10	Advanced database concepts

A quick question:



PollEv.com/joeappleton531

Introduction

Today's outline

- About me
- Module learning outcomes
- Module general information
- Module content
- Introduction to internet systems
 - ✓ Web, URL
- Front-end
 - ✓ HTML
 - ✓ Encoding

About Me

- 10-years start up and industry experience.
- 8-years' experience as an academic
- Research interests are software engineering in the context of learning in HE.

Learning outcomes for this Section

- On successful completion of this module, you will be able to:
 - ✓ Understand **how the World Wide Web works** and how it relates to the wider context of the Internet.
 - ✓ Understand basic concepts of computer networking.
 - ✓ Demonstrate an understanding of **key Web front-end (publishing) technologies** and their relationships: HTML, CSS, Node.js, XML, JSON.
 - ❖ **Build** static and simple dynamic web pages.

Some of the technologies we'll be using

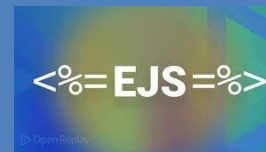
Client



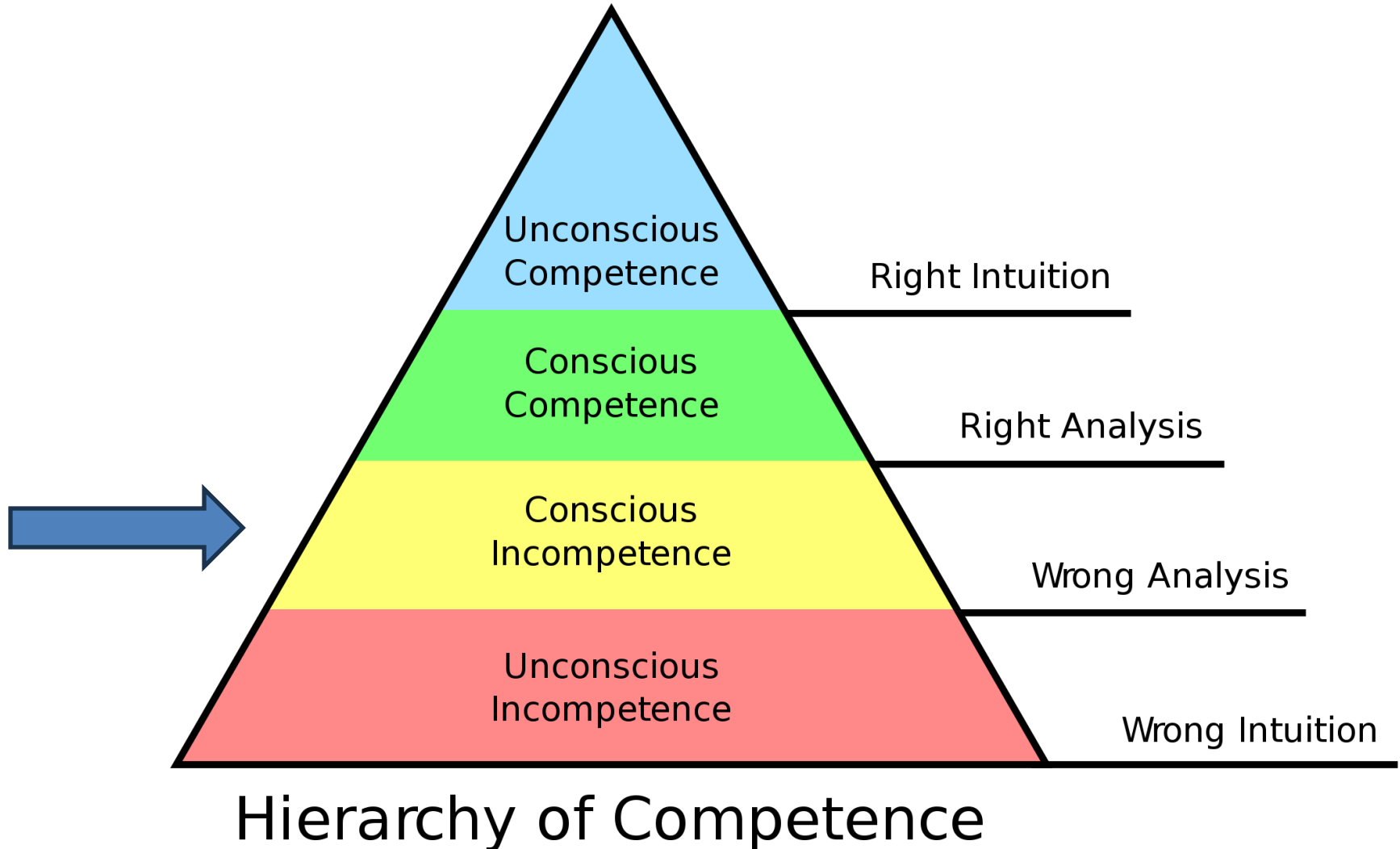
Server



express



Where are we aiming?



Module follow-ups

- COM1025 was designed to be an entry-level module
 - ✓ Cover many topics
 - ❖ JavaScript, CSS
 - ✓ More advanced materials are available for year-2 and/or year-3
- COM2025 Web Application Development
 - ✓ ...
 - ✓ JavaScript (jQuery)
 - ✓ CSS (LESS)
 - ✓ ...
- COM3014 Advanced Challenges in Web Technologies
 - ✓ ...
 - ✓ Rich Client Web Applications
 - ❖ JavaScript
 - ✓ ...

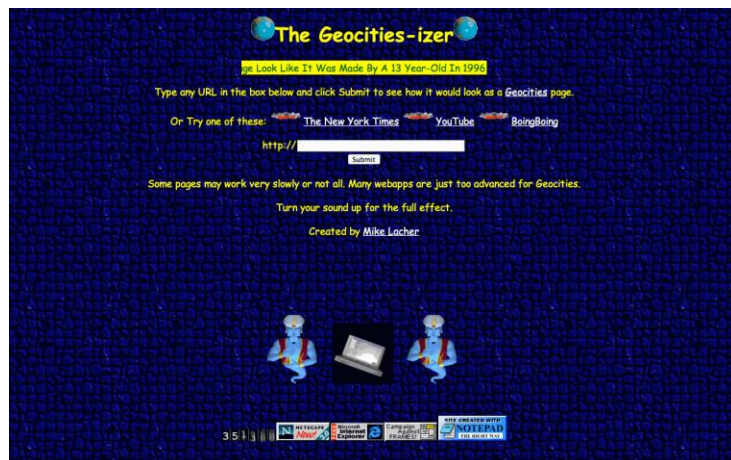
The World Wide Web

A Trip to the 80s



An early timeline of the web

- 1989: Tim Berners-Lee proposed a hypertext system that would sit on top of the internet.
- The system consisted of the following primary technologies:
 - A Universal Resource Locator (URL) to identify resources
 - Hyper Text Markup Language (HTML) to represent HTML documents
 - A Hyper Text server implementing HTTP
- 1993: The proposal of an IMG tag and the Mosaic Browser
- 1994: [Yahoo! is created](#) by Stanford University Graduates
- [GeoCities Created](#)
- 1995: Amazon and Ebay are created



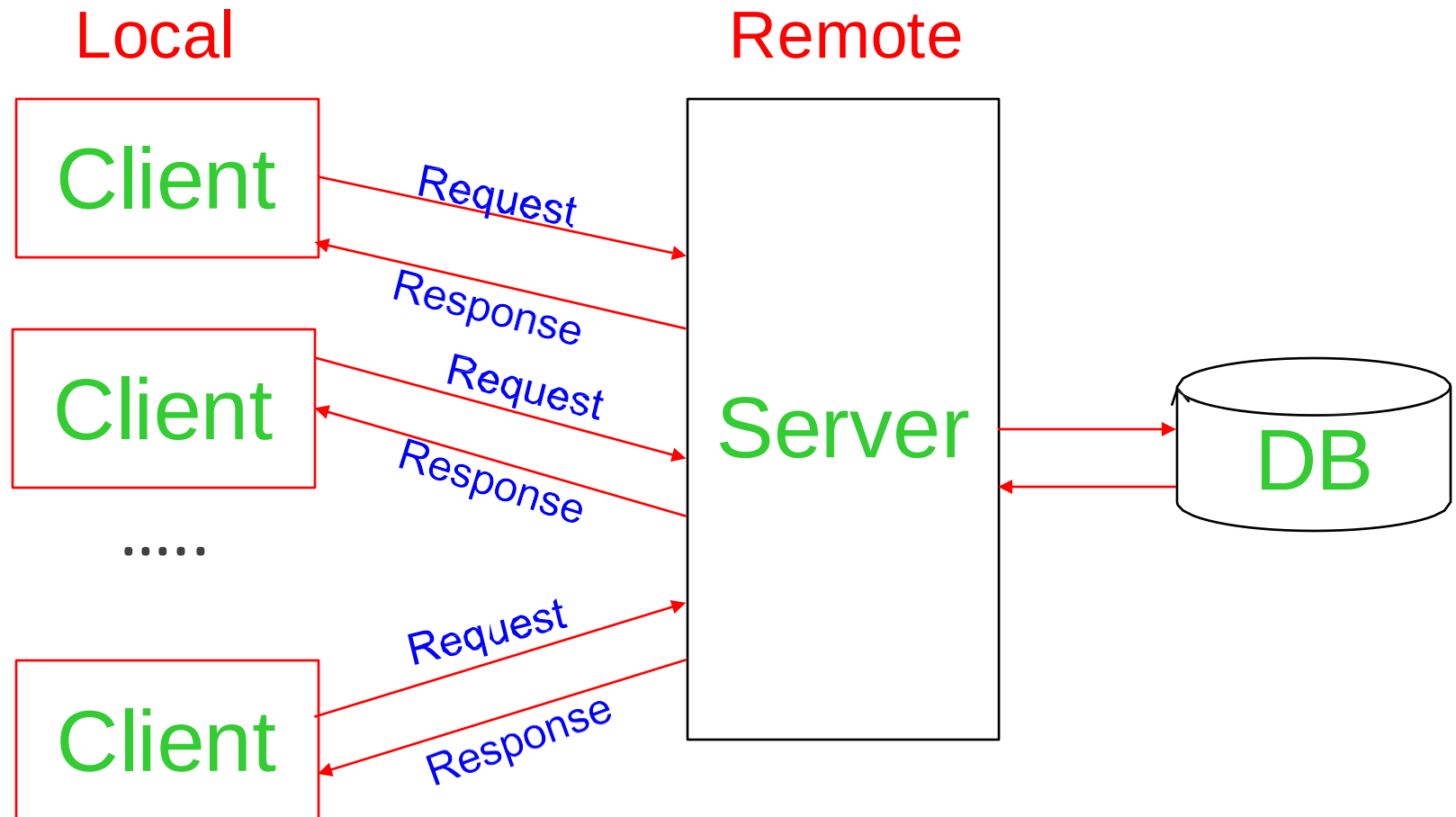
What is HTML?

■ HTML (HyperText Markup Language)

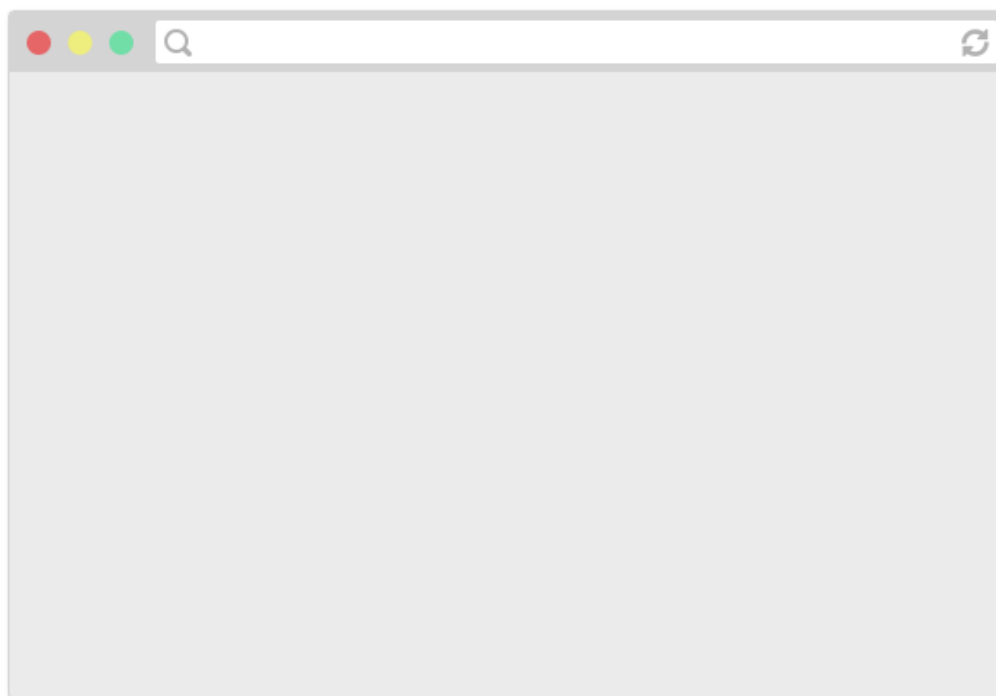
- ✓ Describes the **content** and **structure** of a web page; not a programming language.
- ✓ Made up of building blocks called **elements**.
- ✓ HTML was invented, in 1989, by Tim Berners-Lee (Prof Univ. of Oxford)
- ✓ Standards are maintained and developed by the [World Wide Web Consortium \(W3C\)](http://www.w3.org/).



Client Server



How do web pages work?

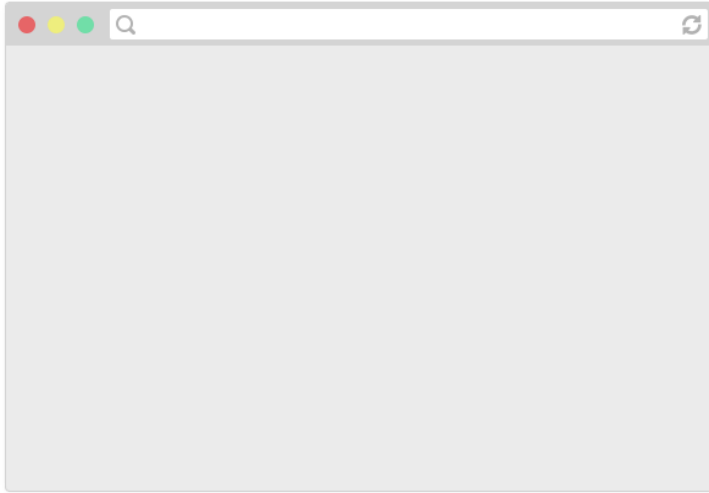


- **Browsers** are applications that can **display web pages**, e.g. Chrome, Firefox, Safari, Internet Explorer, Edge, etc.

Hypertext documents

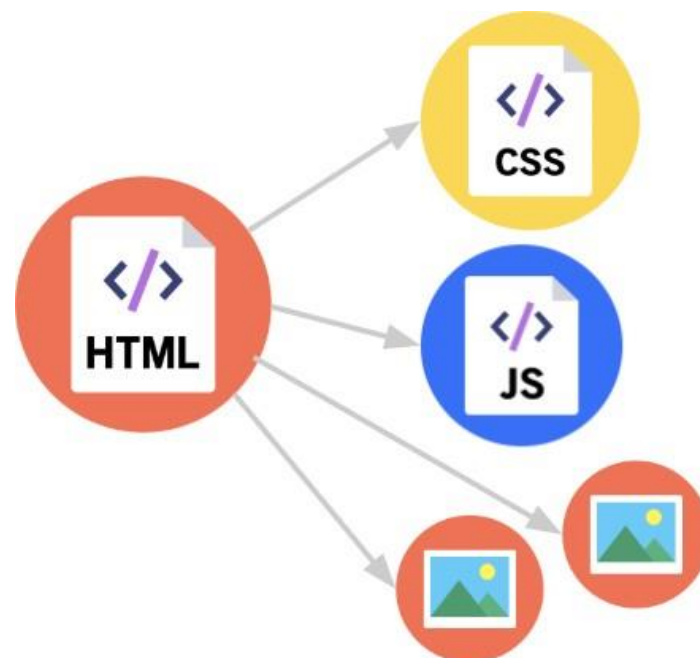
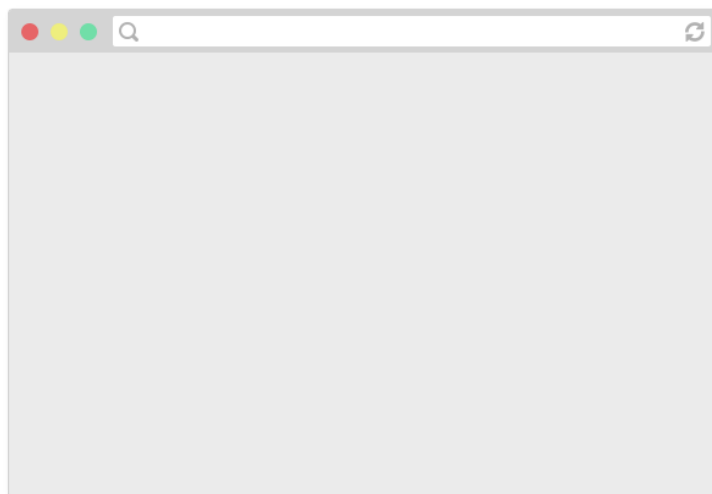
- Hypertext documents are more commonly known as web pages.
- Hypertext documents may link to hypermedia documents, i.e., non-hypertext documents (e.g. an image, a video, a textual file without any special structure).
- A web page or a hypermedia document normally resides on a remote computer (called a web server) over the Internet so they can be accessed by people from anywhere in the world.
 - ✓ Each web page or hypermedia document has a web address, or more formally a URL (Uniform Resource Locator) which mainly tells where it is.

How do web pages work?



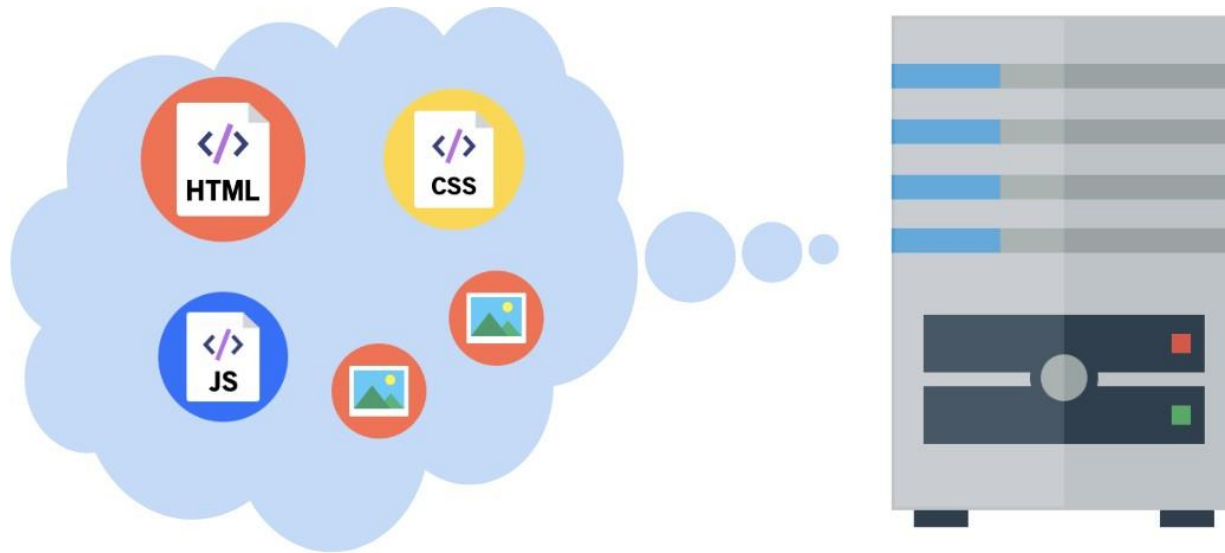
- Web pages are written in a **markup** language called **HTML**
- Browsers **display** a web page by **reading** and **interpreting** its HTML

How do web pages work?



The HTML file may link to other resources, like images, videos, as well as **JavaScript** and **CSS** (stylesheet) files, which the browser then also loads.

How do web pages work?



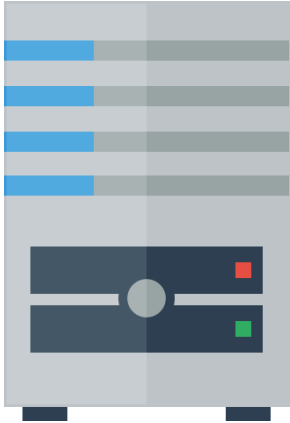
- A **web server** is a program running on a computer that delivers web pages in response to **requests**.
- It either **has stored** or **generates** the web page returned.

How do web pages work?

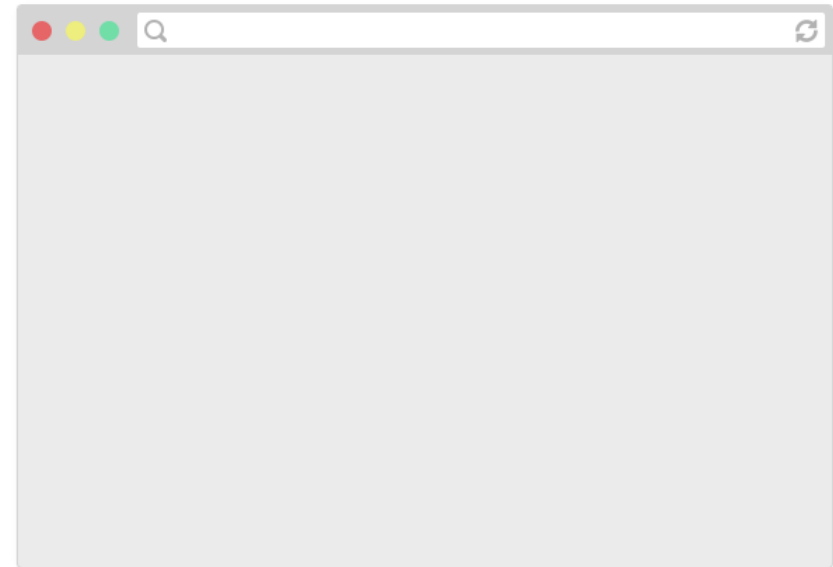
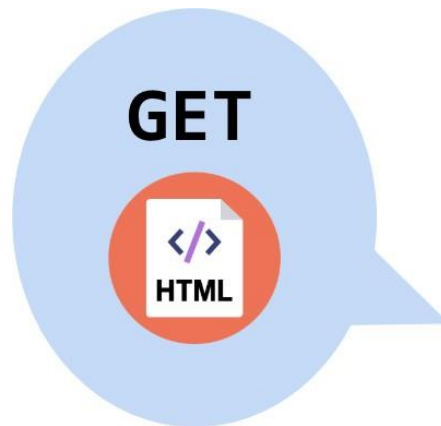
- 1. You type in a URL, which is the address of the HTML file on the internet.



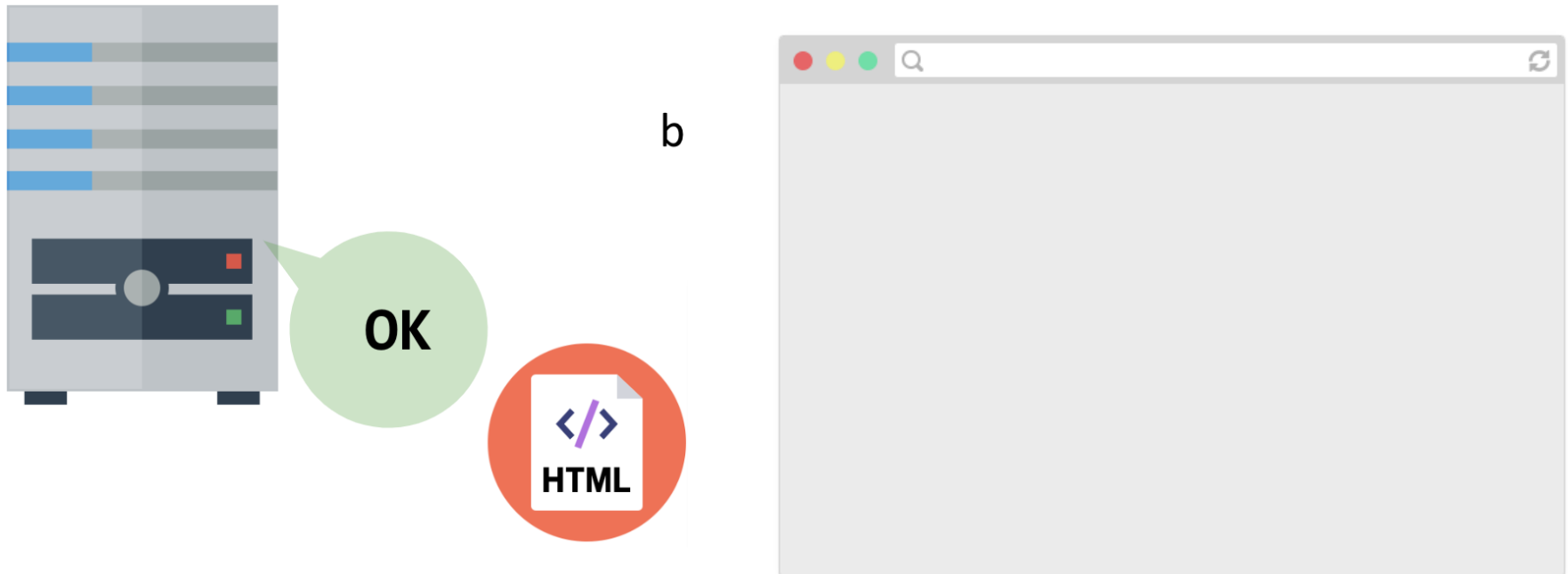
How do web pages work?



2. The browser asks the web server that hosts the document to send that HTML document.

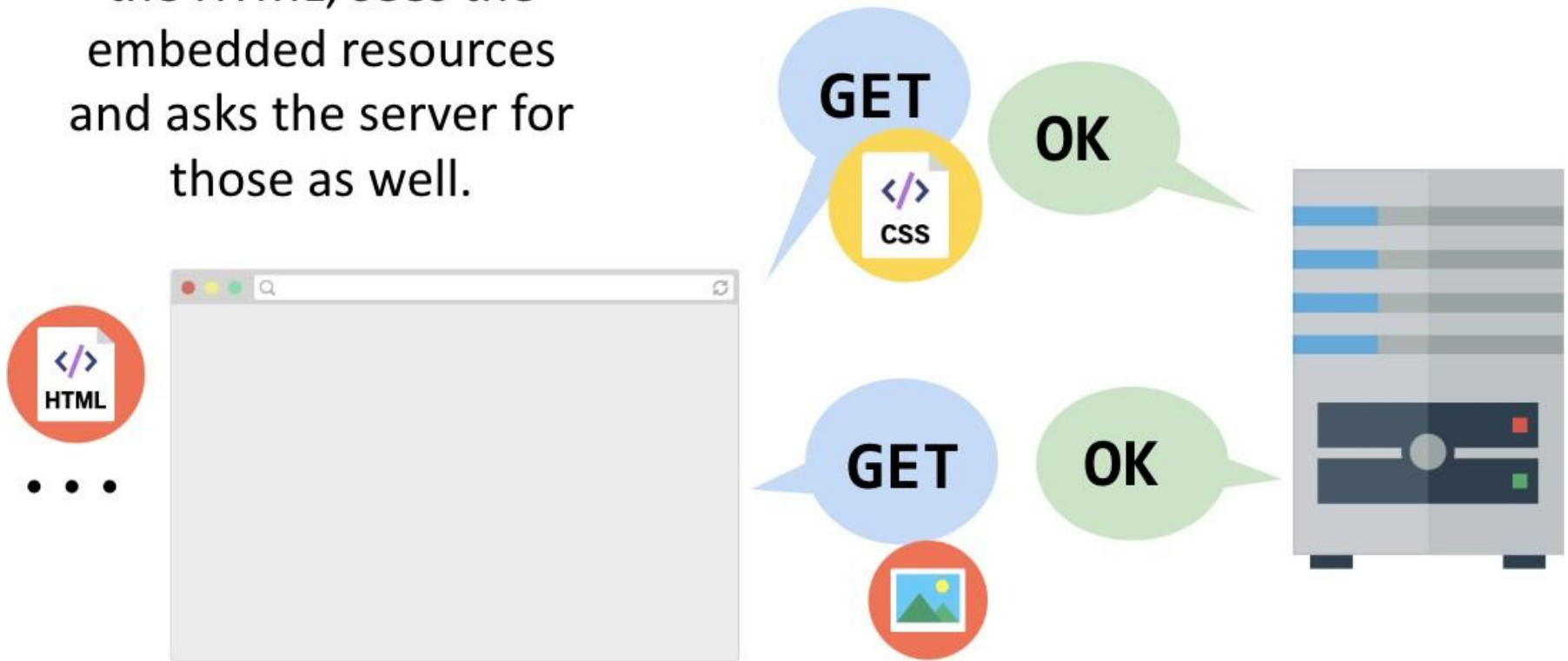


3. The web server responds to the browser with the HTML file that was requested.



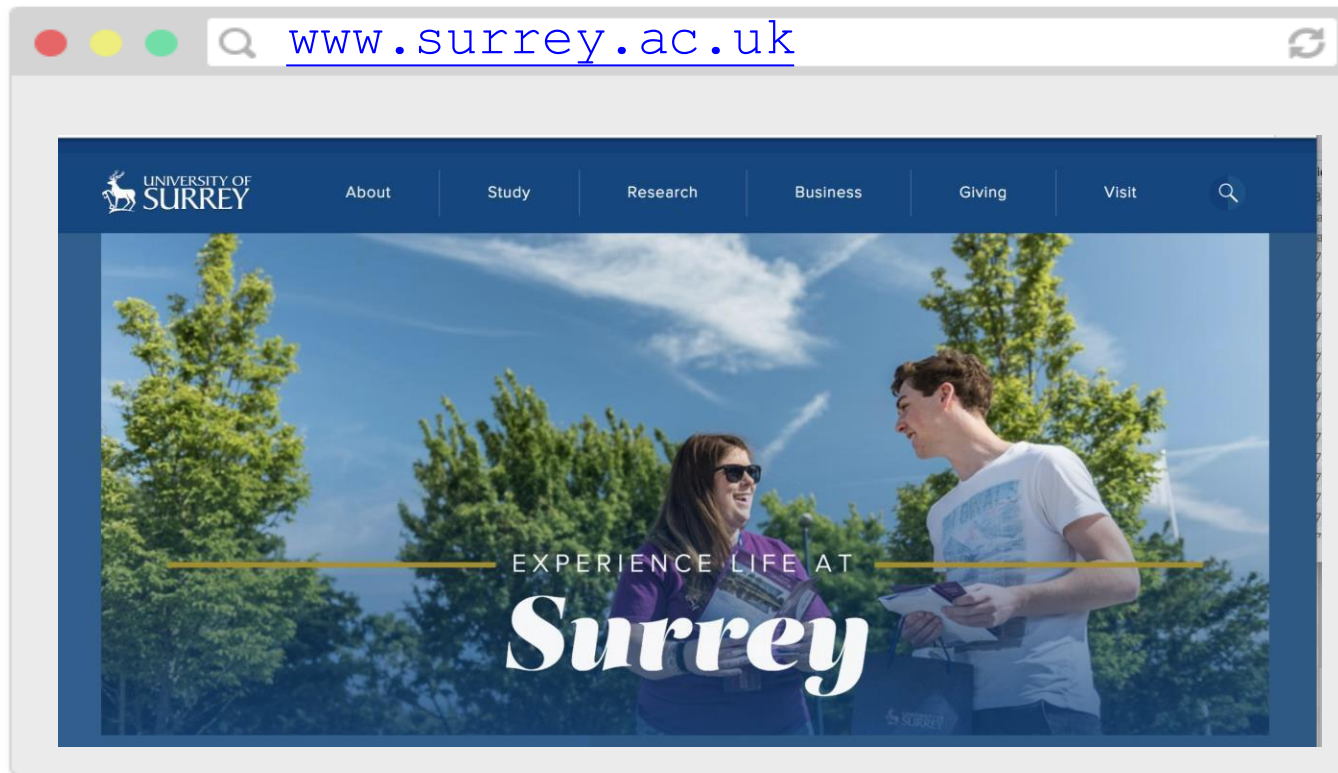
How do web pages work?

4. The browser reads the HTML, sees the embedded resources and asks the server for those as well.



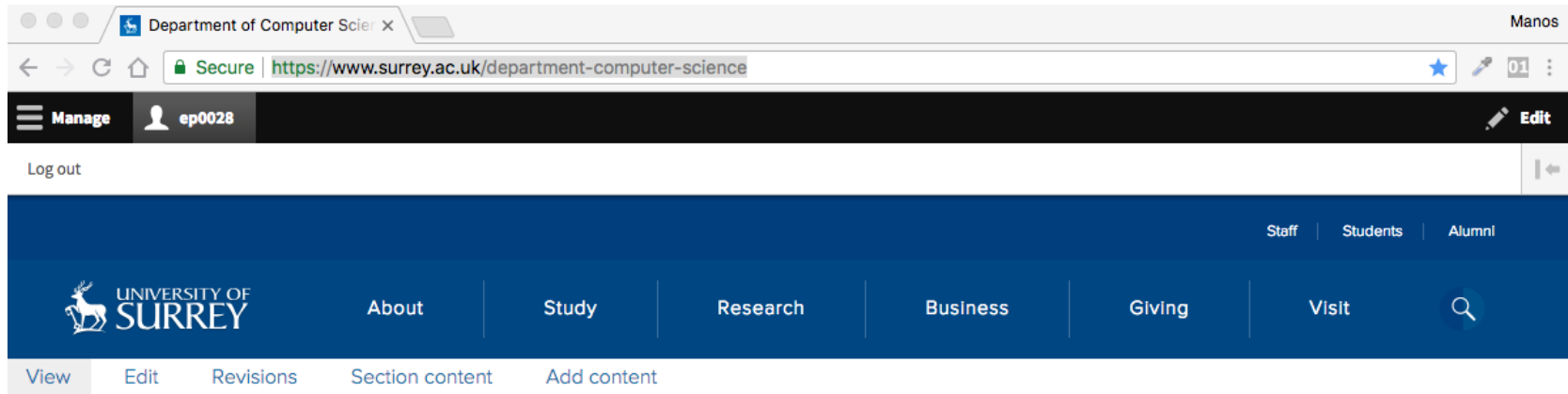
How do web pages work?

5. The web page is loaded when all the resources are fetched and displayed.



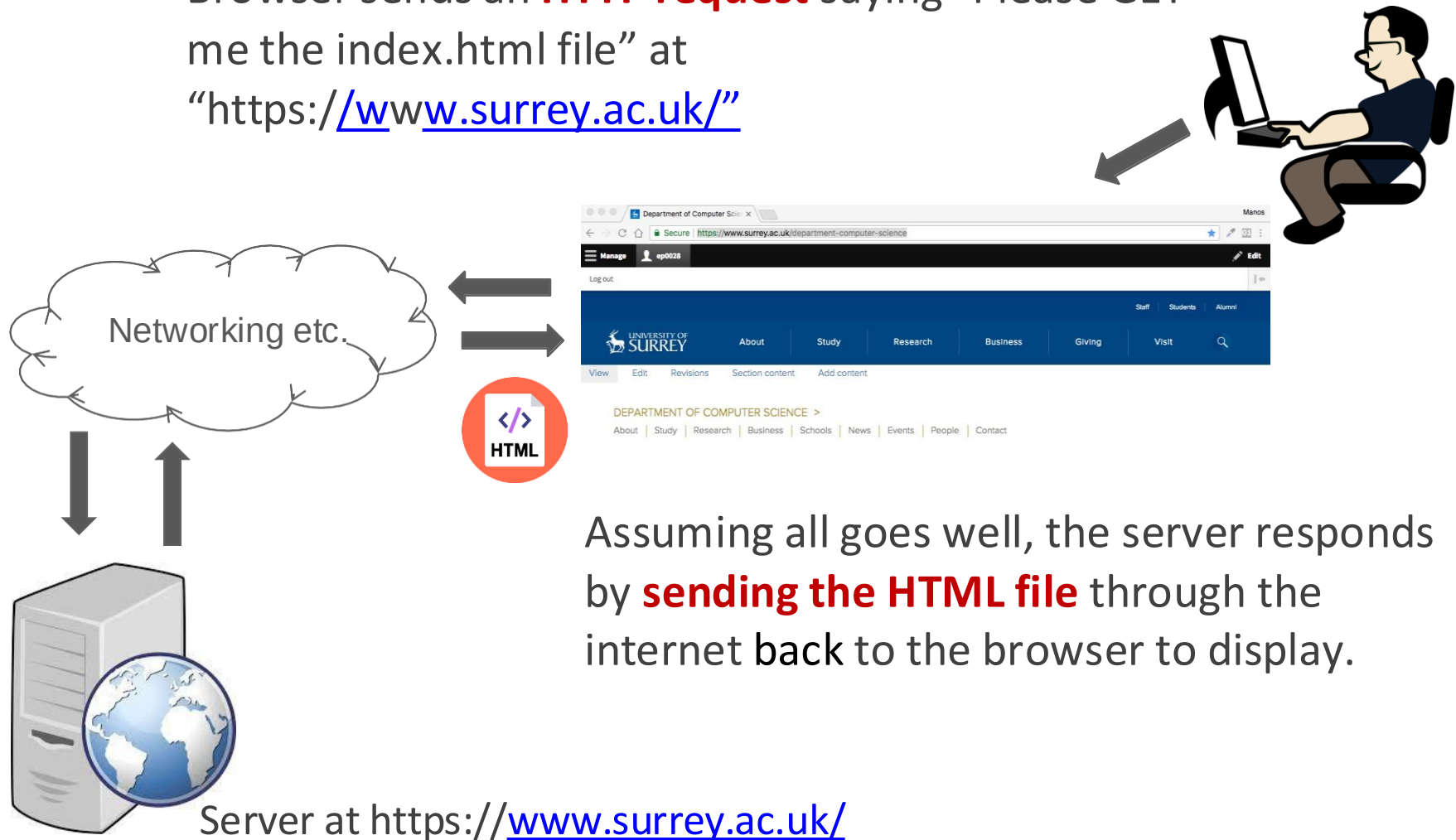
Request – response in more detail

Your PC is running
a web browser



Web browser

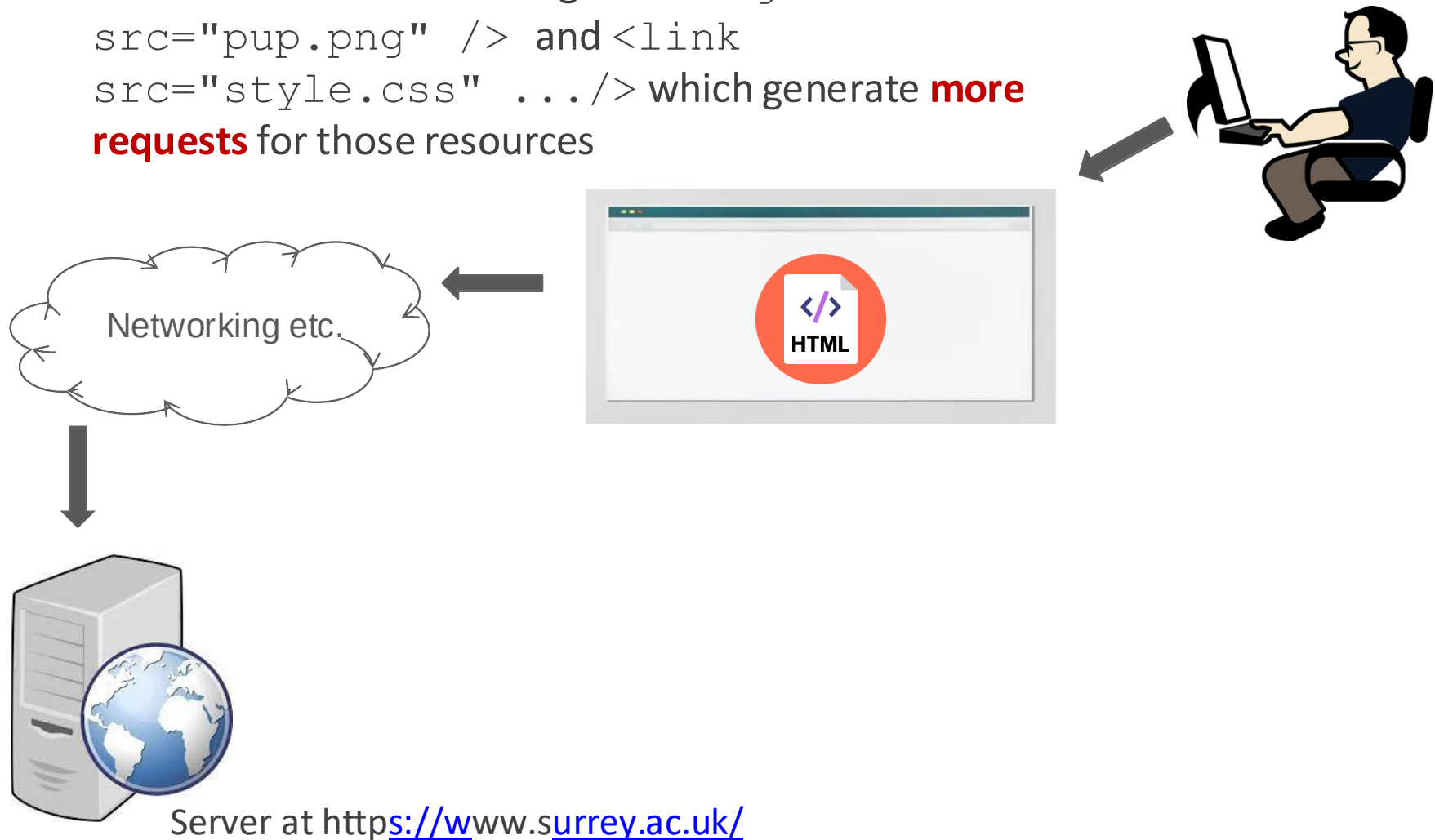
Browser sends an **HTTP request** saying "Please GET me the index.html file" at "<https://www.surrey.ac.uk/>"



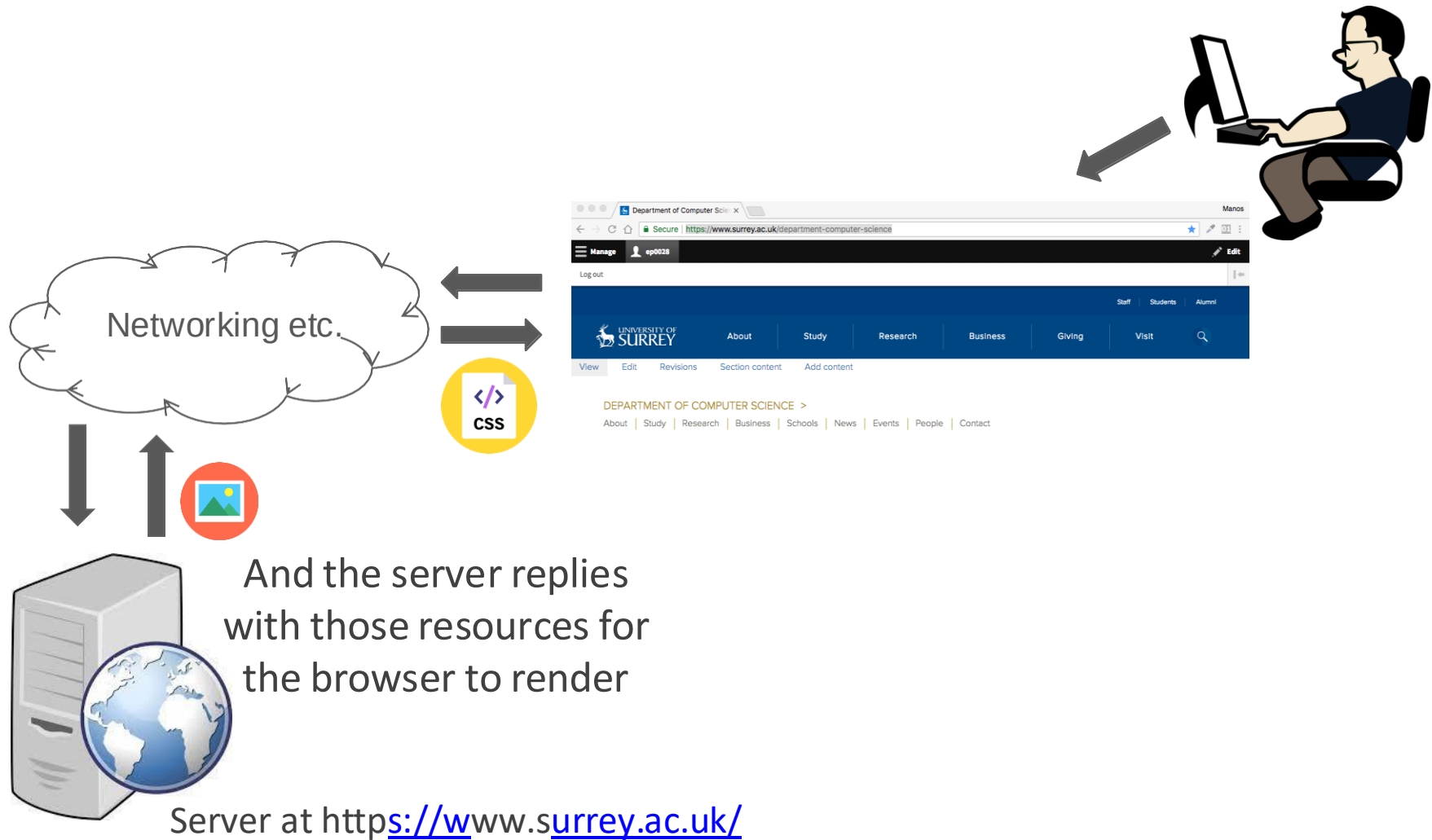
Assuming all goes well, the server responds by **sending the HTML file** through the internet back to the browser to display.

Web browser

The HTML will include things like `` and `<link src="style.css" .../>` which generate **more requests** for those resources



Web browser



Web browser

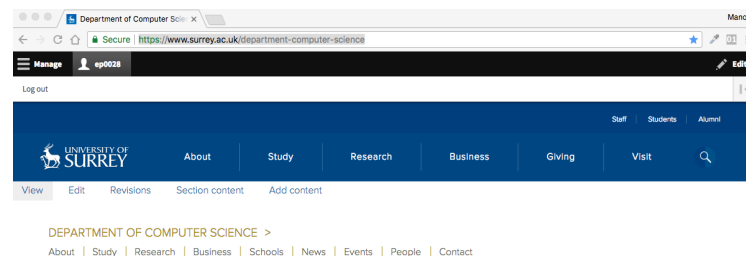


Describes the **content** and **structure** of the page

+



produces



A web page...
that **doesn't do anything**

URL

URI

- A Uniform Resource Identifier (URI) is a **string of characters** used to **identify a resource**
- The most common form of URI is the Uniform Resource Locator (URL), frequently referred to informally as a **web address**, e.g. <https://developer.mozilla.org/en-US/>
 - ✓ Web browsers request pages from web servers by using a URL
- Some URIs refer to a location within a resource.
 - ✓ This kind of URI ends with "**#**" followed by an anchor identifier (called the **fragment identifier**) – see next.

Absolute vs Relative

- You can use both absolute and relative URLs.
 - ✓ Absolute URL: <https://www.surrey.ac.uk>
 - ✓ Relative URL: **about** is relative to <https://www.surrey.ac.uk/about>
- Why does this matter?

```
<div class="container supplementary-global-menu">
  <div class="row">
    <div class="col d-flex justify-content-end">
      <ul>
        <li class="menu-item menu-staff">
          <a href="/staff-information">Staff</a>
        </li>
        <li class="menu-item menu-students">
          <a href="https://my.surrey.ac.uk/">Students</a>
        </li>
        <li class="menu-item menu-alumni">
          <a href="/alumni" data-drupal-link-system-path="node/105361">Alumni</a>
        </li>
      </ul>
    </div>
  </div>
</div>
```

URL syntax (as defined in RFC 3986)

■ Full syntax

✓ `scheme:[//[userinfo@]host[:port]][path][?query][#fragment]`

■ scheme (mandatory for absolute URLs)

✓ The approach (communications protocol, “protocol” for short hereinafter) for retrieving the document.

❖ `http`: documents retrieved via a protocol called HTTP.

❖ `https`: documents retrieved via a protocol called HTTPS.

❖ `file`: documents accessible from the local machine.

❖ `ftp`: documents retrieved via a protocol called FTP.

❖ `mailto`: documents to be sent to the given email address (emails).

❖ ...

URLs are not only for the Web!

URL syntax (as defined in RFC 3986)

- Full syntax

authority

└──────────┘

 - ✓ `scheme: [// [[userinfo@]host[:port]] [path] [?query] [#fragment]`
- **Two slashes** (//): When the authority component is absent, the path component cannot begin with two slashes.
- `authority`
 - ✓ A naming authority for parsing parts of the URL.
 - ✓ `userinfo` (optional)
 - ❖ An optional authentication section of a user name and password, separated by a colon, **followed by @**
 - ❖ The passing of authentication information in clear text has proven to be a security risk in almost every case where it has been used.

URL syntax (as defined in RFC 3986)

- authority
 - ✓ host (mandatory)
 - ❖ **Domain name** (or IP address) of the web site (server).
 - ✓ port (optional)
 - ❖ Port **number** used by the scheme (protocol).
 - In programming, it may specify a particular server program on a computer in a network

URL syntax (as defined in RFC 3986)

■ Full syntax

✓ `scheme: [// [userinfo@]host [:port]] [path] [?query] [#fragment]`

All the rest three components are optional.

■ path

✓ Location of the document on the hosting machine (using “/” as the path separator).

■ query also called a query string

✓ A list of data sent to the web server, in the format
“name_1=value_1&...&name_n=value_n”.

✓ Typical URL containing a query string is as follows:

<http://example.com/over/there?name=ferret>

✓ When a server receives a request for such a page, it may run a program, passing the query string, which in this case is, **name=ferret** unchanged, to the program.

✓ The first question mark is used as a separator, and it is not part of the query string.

■ fragment

✓ ID of a fragment (section/part) of the document (“pointer to an HTML element”).

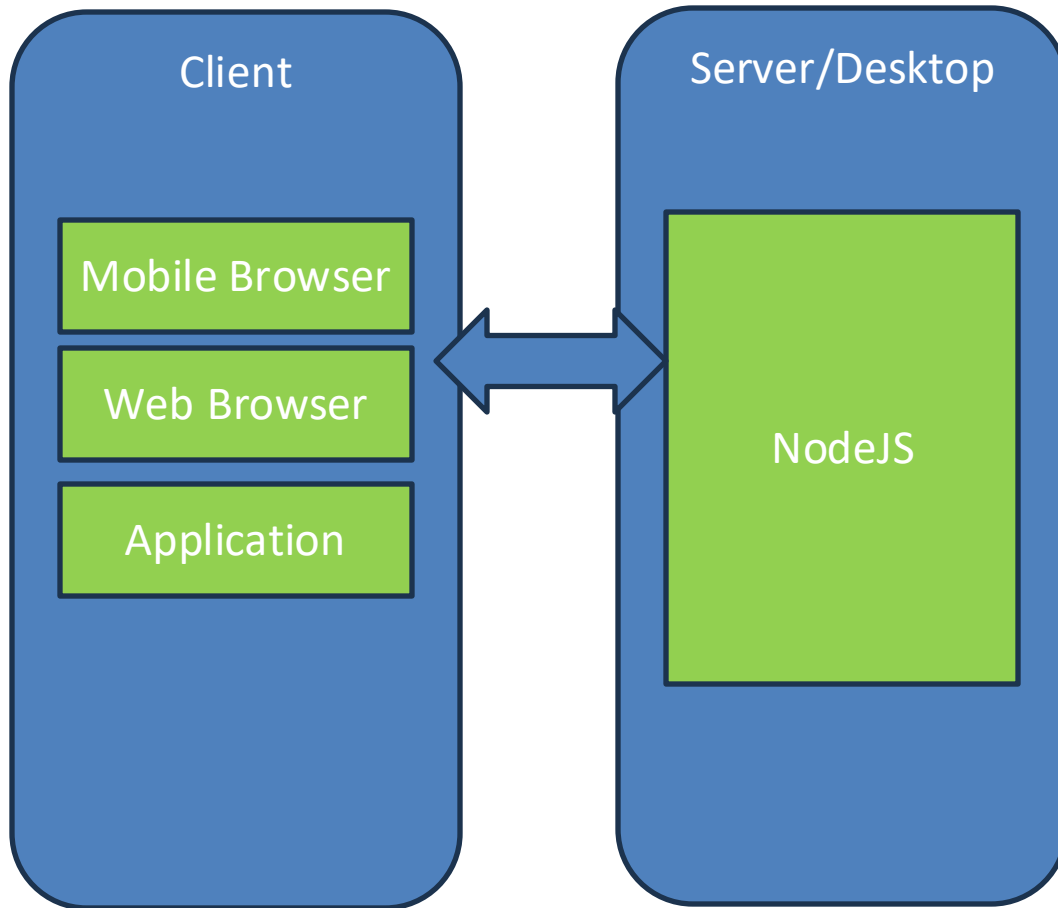
✓ <http://www.example.org/foo.html#bar> the fragment refers to **the element with id="bar"**.

JavaScript

Defining Some Terms

- **JavaScript:** JavaScript is a programming language primarily used for creating interactive and dynamic content within web browsers. **You won't be required to write JavaScript that runs in the web browser, for this module.**
- **Node.js:** : Node.js is not a programming language but a runtime environment that allows you to run JavaScript code on the server. **This is the environment we will be using for this module**

Where Does Node.JS Run

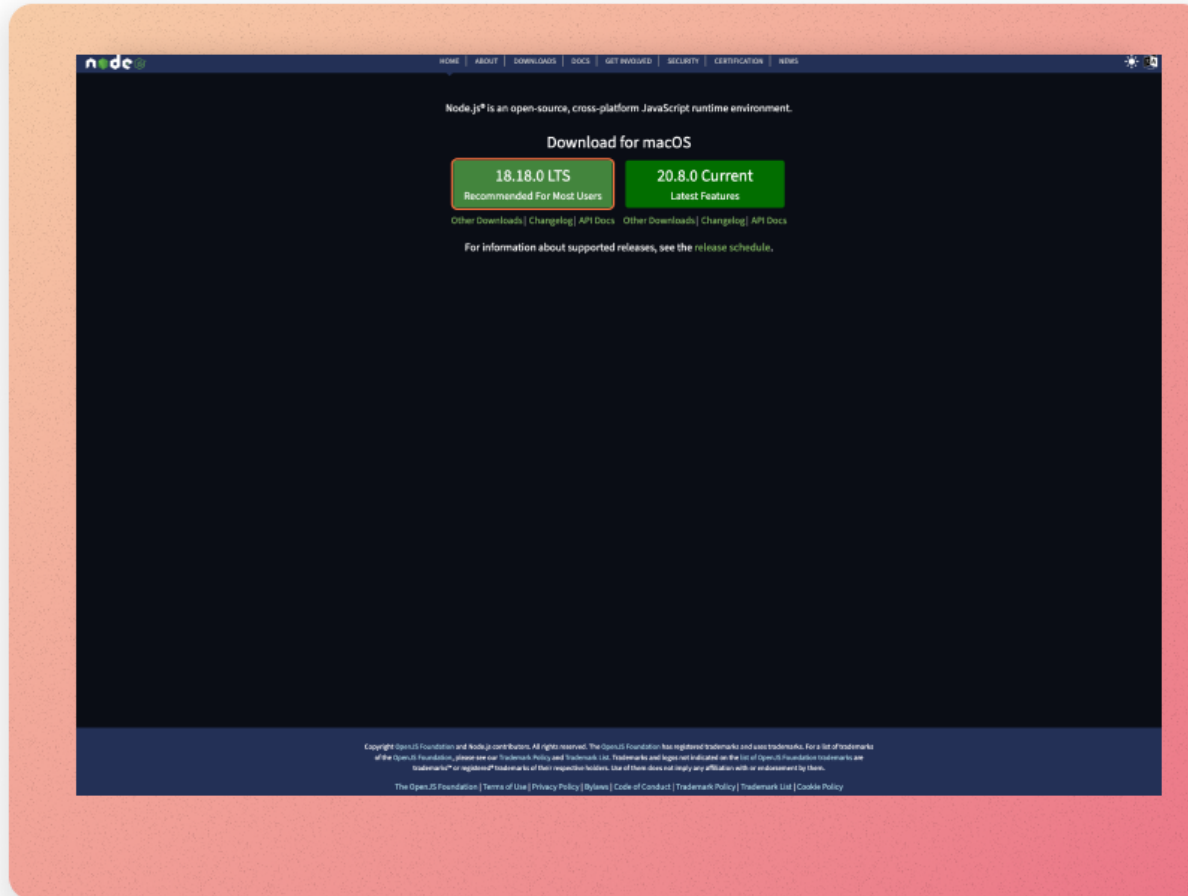


- runs on a server or desktop computer
- It can handle HTTP requests and responses
 - i. making it well-suited for building web applications

Who Uses Node.JS?



Installing Node.JS



To run JavaScript in the NodeJS environment, you should install NodeJS on your computer. Go to the official Node.js website at <https://nodejs.org> and download the installer

Creating Our First Node.js Program

```
1 console.log("hello world");  
2
```

index.js

```
→ node-app node index.js  
hello world  
→ node-app █
```

Our terminal

HTML

Part I

HTML and CSS strategy

- **Assumption:** Most people have cursory familiarity with HTML and CSS. Therefore we will:
 - ✓ **Speed through** the obvious stuff
 - ✓ **Skip** self-explanatory syntax
 - ✓ **Skip** the parts you can look up easily through Google

What is a markup language?

- “A text-formatting language designed to transform **raw text** into **structured documents**”

Source: *American National Standard Dictionary of Information Technology* (ANSDIT)

- Source code (**raw text**)
 - ✓ `<p style="color:red;">Red Text<p>`
- What you see in browsers (**structured document**)
 - ✓ Red Text

HTML and CSS

■ HTML

- ✓ a **markup language** designed for web use
- ✓ describes the **structure** and **content** of the page
- ✓ **descriptive** markup

■ CSS

- ✓ a generalised **stylesheet language**
- ✓ specifies **how the page should be rendered** in various media (e.g. visual styles for screen display)
- ✓ **procedural** (presentational) markup

HTML editor?

- HTML documents (web pages) are **human readable textual** files.
- ⇒ You don't need any special software to create/edit HTML documents!
- ⇒ **Any textual editor** will do the job. [I use [VS Code](#)]
 - ✓ However, you may find a textual editor with HTML syntax highlighting (e.g. [Notepad++](#)) very useful.
- There do exist WYSIWYG (wiz-ee-wig) HTML editors (e.g. [Adobe Dreamweaver](#)), but to learn you better work directly with raw HTML code!

Web server or local files?

- Web pages do not have to be tested on a remote web server.
- To start learning HTML, you only need **a (good) textual editor and a (good) web browser** on your local machine!

HTML tags

- HTML **tags** = Keywords surrounded by angle brackets, e.g. `<html>` and `<p>`
- HTML tags normally come with pairs, e.g. `<html>` ... `</html>` and `<p>` ... `</p>`.
 - ✓ In each pair of HTML tags, the first one (e.g. `<html>`) is called the **start tag** or **opening tag**, and the second one (e.g. `</html>`) is called the **end tag** or **closing tag**.
- Inside the angle brackets, the ending tag starts with a special character “/” (slash).

Basic HTML page structure

Metadata that
doesn't appear
in the viewport
of the browser

Contents that
render in the
viewport of the
browser

```
<!DOCTYPE html>
<html>
  <head>
    <title>COM1025</title>
  </head>
  <body>
    ... contents of the page...
  </body>
</html>
```

E.g.
<title>
shows up as
the name of
the tab

Saved in a *filename.html* file

Encoding

Encodings (i.e. character sets)

- Encoding is necessary so that computers understand the data we enter.
 - ✓ Characters must be encoded to a numerical (binary, decimal, hex) equivalent.
- Every character is “given” (i.e., encoded to) an “x-bit number”, called **code point** where x depends on the encoding we use:
 - ✓ e.g.: UTF-32: each character is stored in 4 bytes (=32 bits).
- **URL encoding** converts characters into a format that can be transmitted over the Internet.
- Your **browser will encode input**, according to the character-set used in your page.
- The default character-set in HTML5 is UTF-8 (most popular).

Why do they matter for web content?

- Why do they matter for the Web?
 - ✓ You know it matters for URLs (addresses of web pages).
 - ✓ It matters for **web content** as well...
- Web pages are **textual** documents.
 - ✓ A textual document = **A sequence of characters encoded using a specific character encoding.**
- $\Rightarrow$ A character encoding is needed for creating the content of a web page and **storing** the web page as a file in the web server's file system.
- To display an HTML page correctly, **a web browser must know the character set used in the page.**

Declaring character encoding

- The **Character set** meta tag is used to specify the encoding, of data that your HTTP server sends.
- To display an HTML page correctly, a web browser must know the character set used in the page.
 - ✓ This is specified in the `<meta>` tag:
 - ❖ ~~HTML 4.01: `<meta http-equiv="Content-type" content="text/html; charset=...">`~~
 - ❖ HTML5: `<meta charset="UTF-8">`
- **Why do we need HTML codes?**
 - ✓ The browser may get the encodings wrong.
 - ✓ One symbol might show up as another, or as an unknown character.
- **What is `<meta>`?**
 - ✓ `<meta>` elements declare different kinds of metadata about an HTML document. (We will come back to it later.)
 - ✓ Metadata will not be displayed on the page but will be **machine parsable**.
 - ✓ `<meta>` elements should be nested in the `<head>` element.

HTML (part II)

HTML vs CSS

- HTML:

- ✓ provides a structure that defines **elements** like tables, forms, lists and headings, and identifies where different portions of our content begin and end
- ✓ can be used to embed other file formats like videos, audio files, documents like PDFs and spreadsheets, among others

- CSS:

- ✓ stands for **cascading style sheet**
- ✓ used to create rules about the colour, font, and layout of our pages
- ✓ also determines when those rules are to be used, based on information like the device connecting to the page, or in response to a user's action

Page Layout

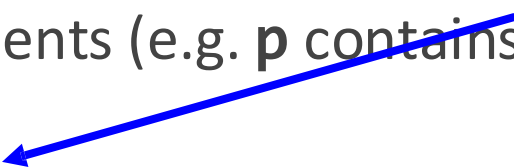
- To add content to our page, we will set up our file with some basic structure
- We use tags to identify information about where parts of our page start and end
- We will do this by using our first set of tags, `<html>` and `</html>`
 - ✓ An HTML **element** is everything from a start tag to the corresponding end tag: `<title>This is the title.</title>`
 - ✓ HTML elements are building blocks of HTML documents
 - ✓ **Wrapping** = placing a set of tags around content and other code
- The **content** of an HTML element is everything between the start tag and the end tag.

Page Layout

- HTML elements can be **nested**:

```
<head><title>Title</title></head>
```

- An element can be self-closing (**img**)
- An element can have attributes (**src="puppy.png"**)
- Elements can contain other elements (e.g. **p** contains **em** and **img**)



```
<p>  
  HTML is <em>awesome!!!</em>  
    
</p>
```

Some HTML elements

(to place within `<body>`)

Top-level heading h1, h2, ... h6	<code><h1>Headline</h1></code> which one is larger?
Paragraph	<code><p>The cat sat on the mat.</p></code>
Line break	First line <code>
</code> Second line
Image	<code></code>
Link	<code>click here!</code>
Strong (bold)	<code>Be BOLD</code>
Emphasis (italic)	My research field is <code></code> machine learning <code></code> and <code></code> computer vision <code></code>
Horizontal rule	My Bio: ... <code><hr></code> My Publications

Some HTML elements

Image: **img**

```

```



Link: **a** (note: not **link**)

```
<a href="google.com">click here!</a>
```

[click here!](https://www.google.com)

Strong (bold): **strong** (note: don't use **b**)

```
<strong>Be BOLD</strong>
```

Be BOLD

Emphasis (italic): **em** (note: don't use **i**)

```
He's my <em>brother</em> and all
```

He's my *brother* and all

Types of HTML elements

- Each HTML element is categorised by the HTML spec into:
 1. **block**: large blocks of content, has height and width
`<p>`, `<h1>`, `<blockquote>`, `<ol>`, `<ul>`, `<table>`
 - ❖ Starting with a **new line**
 - ❖ Take up the full width of the page (**flows top to bottom**)
 - ❖ Have a height and width
 - ❖ Can have block or inline elements as children
 2. **inline**: small amount of content
`<a>`, `<em>`, `<strong>`, `<br>`
 - ❖ Starting within the **current line**
 - ❖ **Cannot** have height and width
 - ❖ **Cannot** have a block element child
 - ❖ **Cannot** be positioned (i.e. CSS properties like float and position do not apply to inline elements)
 3. **inline block**: inline content with height and width `<img>`
 4. **metadata**: information about the page, usually not visible
`<title>`, `<meta>`



Head

- This is where we:
 - ✓ put information about the page itself (called metadata)
 - ✓ connect to any other resources like scripts and files that are not part of the page we are using
- It is important that your `<head>` tags are always the first set of tags after `<html>`, as this is the order in which browsers expect the information
- Add some meta tags inside our head to give it a **title**, **keywords**, a **description**, and other details
 - ✓ These pieces of information help the user and bots understand what the page is about

```
<head>
  <title>Our First Page</title>
  <meta name="keywords" content="beginning html, learning,
COM1025" />
  <meta name="description" content="A beginning web page for
COM1025-Web and Database Systems" />
  <meta name="author" content="Your Name Here" />
</head>
```

HTML attributes

- Most HTML elements have compulsory and/or optional **attributes**
- Attributes provide **additional information** about an HTML element
- Attributes are always specified **in the start tag**
- How an attribute is specified (**name** + **value**):
 - ✓ `name="value" or name='value'`
 - ✓ `name=value` is also allowed if value does not contain space or any of any of `"'`=<>`
 - ✓ the quotes identify what belongs to that attribute
- Example
 - ✓ `<html lang="en">` (`lang` defines language used)
 - ✓ `<meta charset="UTF-8">`

HTML facts

- **Comments in HTML**

- ✓ `<!-- Comments are here. -->`

- The file extension of an HTML document can be either “htm” or “html”

- ✓ No differences apart from the fact that some host servers expect index.html than index.htm

- HTML is case **insensitive**

- In HTML code, more than one continuous spaces count as one space and line breaks are treated as spaces

- ✓ More than one spaces (recall ** **;

Header, Footer

- The header and footer tags are new in HTML5.
- They were added due to the volume of sites that define a top and bottom section to their pages.
- Allowing these tags makes it easier to define and find those parts of the layout.
- The header and footer should be nested within your body tags
- For our example, we will put a screen title in our header and a copyright in our footer:

```
1 <body>
2   <header>
3     <h1>This is our first page!</h1>
4   </header>
5   Hello World
6   <footer>
7     &copy; 2013 Your Name Here
8   </footer>
9 </body>
```

HTML tags: lists

- ordered list

- ✓ `<ol>`
 `<li>Item 1</li>`
 `<li>Item 2</li>`
 `...`
 `</ol>`
 - ✓ `li` = list item

- https://www.w3schools.com/tags/tag_ol.asp

HTML tags: lists

- unordered list

```
✓ <ul>
    <li>Item 1</li>
    <li>Item 2</li>
    ...
</ul>
```

- https://www.w3schools.com/tags/tag_ol.asp

HTML tags: lists

- description list

- ✓ `<dl>`

- `<dt>item1</dt>`

- `<dd>dscrp1</dd>`

- `<dt>item2</dt>`

- `<dd>dscrp2</dd>`

- `</dl>`

- https://www.w3schools.com/tags/tag_dl.asp

Tables

- Tables are a method of formatting the content of your page, and are very similar to the concept of a spreadsheet
- To create a table, we first need to define its beginning and end with tags:
 - ✓ `<table></table>`
- Adding row `<tr>`, Add table heading `<th>`
- https://www.w3schools.com/tags/tag_table.asp

HTML tag: <iframe>

- Basic syntax
 - ✓ `<iframe src="..." ...>text for web browser without
iframe support</iframe>`
- This HTML tag allows display of another HTML document in an inline (embedded) frame.
- Some commonly used attributes
 - ✓ src: URL of the embedded HTML document
 - ✓ name: name of the iframe (which can be the value of `target` attribute of an `<a>` element)
 - ✓ width, height: width and height
 - ✓ frameborder: size of the iframe border
- https://www.w3schools.com/tags/tag_iframe.asp

End of Week 1